

Kairos: A Hybrid Delta/Window-Attention Mixture-of-Experts Denoising Model over a Byte-Level Multimodal Latent Space Design, Compute Sizing and a Tiny-Scale Proof of Concept

Fabien Furfaro*

September 2026

Abstract

Kairos is a proof-of-concept multimodal denoising model that combines bidirectional gated-DeltaNet linear attention and bidirectional sliding-window attention over shared projections (LiZAttention2), block-wise Attention-Residual aggregation, a sparse Mixture-of-Experts (MoE) feed-forward layer, and a multi-scale byte-level codec, trained with a uniform-noise denoising objective over a shared byte-level space for text, image, audio, video, lidar and robot-control streams. We give code-derived algorithms for each component, including a cross-session recurrent-state memory gate. We then size a 200M-total/25M-active target with a roofline model of sparse training: because a microbatch touches nearly every expert, weight traffic scales with the *total* parameter count, so the microbatch that makes a sparse model compute-bound is $N_{\text{total}}/N_{\text{act}}$ times larger than a dense model’s (about 1.6k–2.4k tokens on T4, A100 and H100 for our design point, a factor of 8). Past that threshold the target needs 4.5×10^{18} training FLOPs for 30B tokens; the throughput estimates are upper bounds and none is measured. Finally, we report a tiny-scale ablation study (1–15M parameters, text only) whose apparent architectural effects turn out to be convergence-speed effects on a memorization task: the shared-versus-separate attention-projection gap closes with more steps or diverse data and reverses when the suspected mechanism is controlled. No training run at target scale and no multimodal result is reported.

1 Introduction

1.1 Background and problem statement

Small, edge-deployable language models (MobileLLM [?], SmolLM2 [?], Gemma 3 [?]) are routinely trained on far more tokens than the Chinchilla-optimal ratio [?], on the premise that inference cost dominates total cost of ownership. That premise says nothing about training cost when the training hardware is itself constrained, the setting of this report. Three architectural levers target efficiency in largely separate literatures: (i) sub-quadratic sequence mixing, through local windowed attention [??] and linear state-compressing mixers derived from state-space models (Mamba [?]) and the delta rule [?]; (ii) sparse capacity scaling through MoE routing [????]; (iii) non-autoregressive discrete diffusion training [????], which generalizes masked language modeling [?]. A fourth, orthogonal choice removes the tokenizer: byte-level inputs [??] give one vocabulary that can cover several modalities.

Each lever is usually reported in its own currency (asymptotic sequence complexity, FLOPs per token, or generation parallelism), and rarely against the memory-bandwidth limits of the GPUs that motivate combining them. FLOP-centric accounting of an MoE counts active parameters only, but a training microbatch routes tokens to many experts, so nearly every expert’s weights are read and its gradients written at every step. **Research question:** for a hybrid linear-attention/MoE/denoising model trained

*Code and configurations: <https://github.com/fabienfrfr/Kairos>

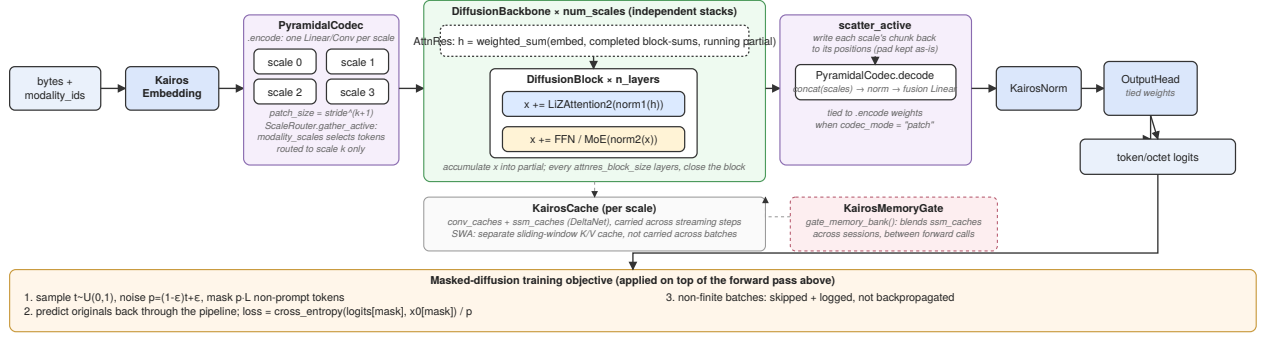


Figure 1: Kairos data and training flow. Byte-level input is embedded, split into `num_scales` branches by the pyramidal codec, and processed by an independent `DiffusionBackbone` per scale: each of its `n_layers` blocks is a standard pre-norm residual pair (attention, then FFN/MoE, each added back to the stream), but the block’s *input* is first re-mixed by `AttnRes` from the embedding and prior block outputs. Scales are then written back, decoded (concat $\rightarrow$ norm $\rightarrow$ fusion), and projected to per-modality logits. Training applies the masked-diffusion objective (bottom) on top of this same forward pass. `KairosCache` carries per-scale state across streaming steps; the cross-session `KairosMemoryGate` (dashed) runs between forward calls, not inside one.

on a single memory-limited GPU, at what microbatch size does sparse training become compute-bound, and which parameter count, active or total, is the right one to reduce when it does not?

1.2 Contributions

1. The Kairos architecture, specified from its implementation: `PyramidalCodec` (Algorithm 1), `LiZAttention2` (Algorithm 2), `Block-AttnRes` (Algorithm 3), sparse MoE (§2.5), a cross-session memory gate (Algorithm 4) and the denoising objective with its curriculum (Algorithm 5), each adapted from a specific prior method or, for the codec and the memory gate, combining existing ideas.
2. A roofline model of sparse training (§2.8) that gives the microbatch threshold above which an MoE is compute-bound, applied to a 200M-total/25M-active target (§3.1); the numbers come from a tested script.
3. A tiny-scale ablation study (§3.2) with 3-seed statistics for codec and attention choices and single-run sweeps for seven further factors.
4. A worked correlation-versus-causation case (§3.2.1): a clear single-run effect that does not survive three controls, and a recommendation to report steps-to-threshold rather than loss at a fixed step.

1.3 Related work

Efficient attention. Full self-attention [?] is quadratic in sequence length. The delta rule [??] improves plain linear attention with an error-correcting state update; Kairos runs it *bidirectionally* (a forward pass and a time-reversed pass, concatenated) to match the context a denoising objective needs. Sliding-window attention [??] is likewise bidirectional here, with rotary position embeddings [?]; the code supports grouped-query attention [?], which the configurations reported here do not exercise. Coupling a delta-rule branch and a window-attention branch through shared projections follows TPTT [?].

Byte-level, hierarchical and multimodal models. ByT5 removes the subword tokenizer [?]; Kairos’ tokenizer subclasses it and extends it to other modalities. MegaByte models bytes with a global model over fixed-size patches and a local model inside each patch [?]; the Byte Latent Transformer forms patches

dynamically from next-byte entropy [?]; Hourglass Transformers shorten and re-expand the sequence inside one stack [?]; Perceiver IO reads arbitrary input arrays, including raw bytes, through cross-attention into a fixed latent array [?]. Kairos’ PyramidalCodec instead keeps S parallel views of one sequence at geometrically growing patch sizes, runs a backbone on each, and fuses them after upsampling. We did not perform a systematic literature search and claim no novelty beyond this combination.

Sparse MoE. MoE architectures [????] decouple capacity from per-token compute. **KairosMoE** subclasses DeepSeek-V3’s routed-expert layer [?] (sigmoid routing scores, shared experts, and load balancing by a per-expert selection bias rather than an auxiliary loss), and **KairosFFN**, used when MoE is disabled, subclasses Qwen2’s dense MLP [?]. Only the expert-weight initialization is patched, because the upstream implementation used leaves those weights uninitialized.

Cross-layer residual aggregation. Block-AttnRes replaces the fixed, unit-weight residual accumulation standard since ResNet [?] with the learned, block-windowed softmax attention over earlier layer outputs of ?, used here as published (§2.4).

Discrete diffusion language modeling. Kairos’ corruption is closest to the uniform-replacement transition of D3PM [?]: corrupted positions are filled with tokens drawn uniformly from the vocabulary, and the model is not told which positions were corrupted, unlike the absorbing mask state of MDLM [?]. The loss is weighted by $1/p$ in the manner of MDLM, with a clip, and trained with self-conditioning [?] and a three-stage curriculum (§2.7).

Recurrent memory. Transformer-XL carries hidden states across segments [?], Titans learn a test-time memory [?], and TPTT retrofits such memory into pretrained transformers [?]. Kairos’ gate is simpler: it seeds the recurrent state of each DeltaNet layer for a new sequence from a learned blend of states left by earlier sequences (§2.6).

Scaling laws and the cost of sparse training. Chinchilla is fitted on dense models from 70M to over 16B parameters [?], so applying it to a model with 25M active parameters is an extrapolation. Joint MoE scaling laws [?] add active parameters, tokens and expert count as variables, and ? study the parameters-versus-FLOPs trade-off at optimal sparsity. These works parameterize loss rather than step time; we model step time with the roofline [?].

2 Methods

2.1 Three scales, kept apart

The codebase is used at three scales that this report does not conflate (Table 1). The **ablation networks** of §3.2 use the library defaults of `configs/ablations/baseline.yaml` (1 layer, dense feed-forward). The **PoC configuration** is the default of the project’s notebook (`notebook/kairos_pretraining.py`): a ~ 14 – 15 M-parameter network with a 7-expert top-1 MoE, used for pipeline sanity checks. The **target** is a 200M-total/25M-active design point used in §2.8 purely to size a future run; it has never been instantiated. Because dense parameters (attention, codec, embeddings) are always active, $N_{\text{act}} = N_{\text{total}}/8 + \frac{7}{8}N_{\text{dense}}$ for $E/k = 8$, and 25M is the lower bound reached when all parameters are routed experts.

Figure 2 shows the data flow shared by all three.

2.2 Byte-level input and PyramidalCodec

Every modality is serialized to bytes: text as UTF-8, other modalities as quantized values written as big-endian byte planes (for example a 16-bit audio sample as a high and a low byte). Each byte is one token,

	Ablation baseline	PoC (notebook)	Target (sizing only)
d_{model} ; heads (head dim)	64; 4 (16)	64; 4 (16)	—
Backbone layers	1	4	—
Codec stride; scales S	5; 4	3; 4	—
Codec mode	conv, 1 channel per group	conv, 8 channels per group	—
AttnRes block size	1	4	—
Feed-forward	dense, $I = 2048$	7 routed ($k = 1$) + 1 shared, $I = 544$	32 routed ($k = 4$)
Backbone shared across scales	no	yes	—
Memory gate	off	on	—
N_{total}	1.98M	$\sim 14\text{--}15\text{M}$	200M
N_{act}	N_{total}	$N_{\text{total}} - 2.51\text{M}$ (Eq. 1)	25M
Training data	16 identical sentences, 600 steps	sanity checks	30B tokens (budget)

Table 1: The three scales this report distinguishes. Dashes: the target design point is specified only by its parameter and token budget. The PoC’s 2.51M is derived from its configuration ($4 \times 7 \times 3 \times 64 \times 544 = 2.92\text{M}$ routed-expert parameters, of which 6/7 are inactive); the instantiated count has not been logged. The ablation networks are far smaller than the PoC, so ablation findings are not statements about the PoC.

Algorithm 1 PyramidalCodec, `mode="conv"`: encode, per-scale backbone, decode

Require: $x \in \mathbb{R}^{B \times L \times d}$; patch sizes $p_1, \dots, p_S$; backbones $f_1, \dots, f_S$

- 1: $h \leftarrow \text{Mixer}_{\text{pre}}(x)$ $\triangleright$ shared channel mixing, computed once
 - 2: **for** $s = 1$ **to** S **do**
 - 3: $h_s \leftarrow \text{GroupedConv1d}(\text{PadToMultiple}(h, p_s); \text{kernel} = \text{stride} = p_s)$
 - 4: $z_s \leftarrow \text{Mixer}_s(h_s)$ $\triangleright \lceil L/p_s \rceil$ positions
 - 5: $\tilde{z}_s \leftarrow f_s(z_s)$ $\triangleright$ active positions only, chosen by the scale router
 - 6: $u_s \leftarrow \text{Trim}_L(\text{Mixer}'_s(\text{ConvTranspose1d}(\tilde{z}_s)))$
 - 7: **end for**
 - 8: **return** $\text{Linear}_{Sd \rightarrow d}(\text{RMSNorm}(\text{concat}(u_1, \dots, u_S)))$
-

and an auxiliary *octet-family* id tags which byte plane a token belongs to. Token and family embeddings are fused by a linear layer; the output head, tied to the token embedding, predicts both the byte and its family.

The PyramidalCodec builds S parallel views of the sequence with patch sizes $p_s = \text{stride}^s$ (PoC: stride 3, $S = 4$, so $p = 3, 9, 27, 81$). A shared rank- r bottleneck mixer ($\text{Conv1d}_{d \rightarrow r} \rightarrow \text{GELU} \rightarrow \text{Conv1d}_{r \rightarrow d}$, $r = \max(8, d/4)$, cost $O(rd)$ against $O(d^2)$ for a dense 1×1 convolution) supplies cross-channel context once; each scale then applies a grouped convolution with kernel and stride p_s and its own mixer (Algorithm 1). The group count is the one knob that trades compute for expressiveness: `channels_per_group=1` (default) is depthwise and cheapest, and raising it toward d approaches a dense convolution. `mode="patch"` replaces the convolution and mixer chain by one $\text{Linear}(p_s d \rightarrow d)$ per scale, decoded with the transposed weight (tied encode and decode). Each scale runs through its own backbone, or through one shared backbone (about 75% fewer backbone parameters for $S = 4$). A scale router lets each modality read only a subset of scales (PoC: text $\rightarrow$ scales 1–2, control 1, image 2–3, lidar 2, audio 3, video 3–4, metadata 4); positions of modalities not routed to a scale bypass its backbone. Decoding mirrors encoding with grouped transposed convolutions and per-scale mixers, then concatenates the scales, applies RMSNorm and fuses them with a linear layer.

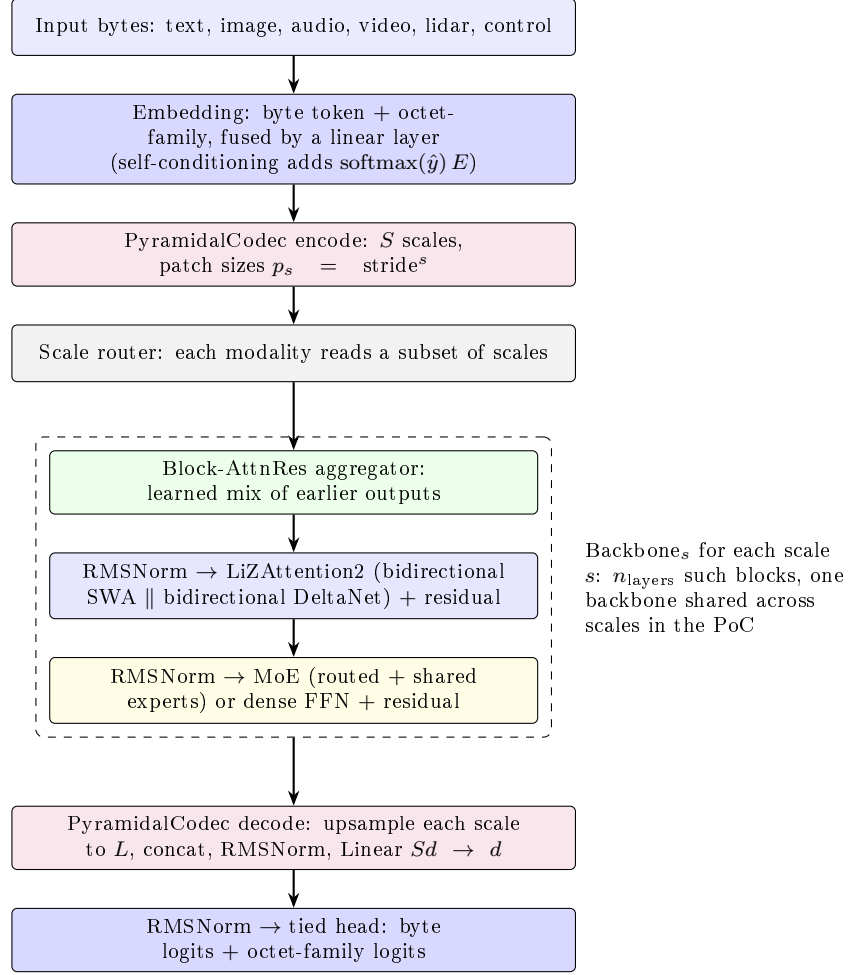


Figure 2: Kairos data flow. The AttnRes aggregator builds the input of each block; a block is a pre-norm residual pair (attention, feed-forward). Scales are fused in the codec decoder, after the per-scale backbones.

2.3 LiZAttention2: shared-projection bidirectional hybrid attention

Each layer runs a bidirectional sliding-window branch (mask $|q - k| \leq w$, RoPE) and a bidirectional gated-DeltaNet branch over the same normalized input. The DeltaNet branch’s Q/K/V and output projections are deleted and replaced by references to the window branch’s, so both mixers read the same linear projections of x (`liz2_share_qkv=False` restores independent ones; §3.2.1). The DeltaNet branch keeps its own value expansion (factor 2), depthwise causal convolution, decay and write-strength gates, output gate, and a merge of its forward and time-reversed passes (Algorithm 2). The two branch outputs are concatenated, normalized with RMSNorm [?] and projected back to d .

2.4 Block-AttnRes aggregation

Instead of the plain residual stream $x_{i+1} = x_i + f_i(x_i)$, each layer’s *input* is a learned, softmax-weighted mix of the embedding, the finalized block sums of earlier layers, and a running partial sum of the current block [?] (Algorithm 3); the sublayers of f_i still use ordinary residual connections around their own input. The weight vector w of each aggregator is initialized to zero, so the initial mix is uniform. With block size 1 each earlier layer output is a separate source, a densely connected pattern [?] with learned softmax weights; with block size n_{layers} the sources reduce to the embedding and one running sum, and larger sizes change nothing. The PoC uses block size 4 over 4 layers; the ablation networks have one layer, where the

Algorithm 2 LiZAttention2 forward (one layer)

Require: $x \in \mathbb{R}^{B \times L \times d}$; shared projections W_q, W_k, W_v, W_o

- 1: $o^{\text{swa}} \leftarrow W_o \text{BidirWindowAttn}(\text{RoPE}(W_q x), \text{RoPE}(W_k x), W_v x; w)$
- 2: **for** $\text{dir} \in \{\text{fwd}, \text{bwd}\}$ **do**
- 3: $x' \leftarrow x$ if fwd, else $\text{flip}(x)$ $\triangleright$ flip along the sequence axis
- 4: $(q, k, v) \leftarrow \text{SiLU DepthwiseCausalConv}([W_q x'; W_k x'; E_v W_v x'])$ $\triangleright E_v$: value expansion
- 5: $\beta \leftarrow \sigma(W_\beta x')$; $g \leftarrow -\exp(A_{\log}) \text{softplus}(W_a x' + b_{dt})$ $\triangleright$ per head
- 6: $o^{\text{dir}} \leftarrow \text{GatedDeltaRule}(q, k, v; g, \beta) \odot \text{SiLU}(W_g x')$ $\triangleright q, k$ L2-normalized
- 7: **end for**
- 8: $o^\Delta \leftarrow W_o W_{\text{lr}} \text{RMSNorm}([o^{\text{fwd}}; \text{flip}(o^{\text{bwd}})])$
- 9: **return** $W_{\text{mix}} \text{RMSNorm}([o^{\text{swa}}; o^\Delta])$ $\triangleright 2d \rightarrow d$

Algorithm 3 Block-AttnRes backbone forward

Require: embedding x_0 ; layers $f_1, \dots, f_L$; block size S ; learned $w_i \in \mathbb{R}^d$ per aggregator

- 1: $\text{completed} \leftarrow []$; $\text{partial} \leftarrow \text{None}$; $\text{count} \leftarrow 0$
- 2: **for** $i = 1, \dots, L$ **do**
- 3: $\text{sources} \leftarrow [x_0] \cup \text{completed} \cup (\{\text{partial}\} \text{ if not None})$
- 4: $V \leftarrow \text{stack}(\text{sources})$; $K \leftarrow \text{RMSNorm}(V)$
- 5: $h \leftarrow \sum_{\text{sources}} \text{softmax}(w_i^\top K) \cdot V$ $\triangleright$ softmax over sources
- 6: $x_i \leftarrow f_i(h)$
- 7: $\text{partial} \leftarrow x_i$ if partial is None else $\text{partial} + x_i$; $\text{count} += 1$
- 8: **if** $\text{count} = S$ **then**
- 9: $\text{completed.append}(\text{partial})$; $\text{partial} \leftarrow \text{None}$; $\text{count} \leftarrow 0$
- 10: **end if**
- 11: **end for**
- 12: **return** $\text{RMSNorm}(x_L)$

aggregator is a structural no-op.

2.5 Sparse Mixture-of-Experts

Each token is routed to k of E routed experts plus one or more always-on shared experts. Balance is maintained by nudging a per-expert selection bias toward equal load (DeepSeek-V3 style, no auxiliary loss by default). The active-parameter count used throughout is computed directly:

$$N_{\text{act}} = N_{\text{total}} - N_{\text{expert}} + N_{\text{expert}} \frac{k}{E}, \quad (1)$$

where N_{expert} sums the parameters under modules whose name contains ".**experts.**"; shared experts, the router, the attention and DeltaNet path and the codec count as fully active (`count_active_parameters`).

2.6 KairosMemoryGate: cross-session recurrent-state memory

With `use_memory_gate=True`, the recurrent state of every DeltaNet layer of every scale is seeded for a new sequence not from zero but from a gate over a bank of states left by earlier sequences (the preceding batch's final state, or an externally supplied bank). One gate module is shared by all scales and layers. It is a single-head cross-attention over a rank- r' bottleneck, $r' = \max(8, \text{round}(\sqrt{D}/4))$, where D is the flattened DeltaNet state size ($D = 2048$ and $r' = 11$ for the PoC). The query is the projection of the (zero) current state; the keys and values are that same projection followed by the projected bank, so the gate can attend to the bank or fall back on the current state (Algorithm 4). It is applied to the forward pass only; the time-reversed pass starts from zero.

Algorithm 4 Memory gate, applied per (scale, DeltaNet layer)

Require: zero state $s_0 \in \mathbb{R}^{B \times D}$; bank $M \in \mathbb{R}^{n \times D}$ of flattened prior states

```
1: if  $n = 0$  then  
2:   return  $s_0$   
3: end if  
4: if  $n = 1$  then  
5:   return  $M_1$  broadcast to  $B$  rows  
6: end if  
7:  $q \leftarrow \text{Linear}_{D \rightarrow r'}(s_0)$ ;  $K \leftarrow [q; \text{Linear}_{D \rightarrow r'}(M)]$   
8:  $o \leftarrow \text{MultiheadAttention}_{h=1}(q, K, K)$   
9: return  $\text{Linear}_{r' \rightarrow D}(o)$ 
```

Algorithm 5 Denoising training step (`compute_loss`)

Require: batch x_0 , prompt lengths, pad mask; schedule values $(p_{\max}^{(t)}, \alpha^{(t)})$; clip c

```
1:  $p_b \leftarrow \epsilon + (p_{\max}^{(t)} - \epsilon) u_b$ ,  $u_b \sim U(0, 1)$  ▷ one rate per row  
2:  $m \leftarrow \text{Bernoulli}(p_b)$  restricted to non-prompt, non-pad positions  
3:  $x_t \leftarrow x_0$  with  $x_t[m] \leftarrow$  tokens drawn uniformly from the vocabulary  
4: if  $u < 0.5$ ,  $u \sim U(0, 1)$  then  $\hat{y} \leftarrow \text{stopgrad}(\text{model}(x_t))$ ; add  $\text{softmax}(\hat{y})E$  to the embeddings  
5: end if  
6:  $\text{logits} \leftarrow \text{model}(x_t, \text{logits\_mask} = m)$  ▷ project masked positions only  
7:  $\ell_i \leftarrow \text{CE}(\text{logits}_i, x_{0,i}) \cdot \min(c, 1 + \alpha^{(t)}(1/p_{b(i)} - 1))$   
8: return  $\text{mean}_i \ell_i$  ▷ + octet-family CE when family ids are present
```

2.7 Denoising objective and curriculum

Given a clean byte sequence x_0 , each row draws a rate $p \sim U(\epsilon, p_{\max})$ and a $\text{Bernoulli}(p)$ mask over non-prompt, non-padding positions. Masked positions are replaced by tokens drawn uniformly from the vocabulary (and random octet-family ids), giving x_t ; no mask indicator is provided. The model predicts x_0 at the masked positions with cross-entropy, weighted per token by $\min(c, 1 + \alpha(1/p - 1))$, a blend between plain cross-entropy ($\alpha = 0$) and full $1/p$ weighting ($\alpha = 1$) in the manner of MDLM [?], clipped at c to bound the variance from rows with small p . With probability 0.5 per batch, self-conditioning [?] feeds a detached, no-gradient first prediction back in as an additive embedding, matching its use in `generate()`. The default is $\epsilon = 10^{-3}$, $c = 3$; an octet-family cross-entropy term with weight 1 is added when family ids are present.

The schedule $(p_{\max}^{(t)}, \alpha^{(t)})$ is a pure function of the global step (`stage_mask_schedule`), so resuming from a checkpoint depends only on the persisted step count. It is flat at $(0.3, 0)$ in the MAE stage, ramps linearly to $(1.0, 1)$ over the transition stage, then stays flat in the diffusion stage. The staging was introduced after an informal observation of optimization instability when sampling the full masking range from a random initialization; that observation is not quantified here, and no run without the curriculum is reported.

2.8 Compute-sizing model

Consider one microbatch of T tokens on a GPU with peak dense half-precision throughput π and memory bandwidth b_s , whose ridge point is $\mathcal{I}^* = \pi/b_s$ [?]. Training costs $6N_{\text{act}}T$ FLOPs [?], ignoring attention-score FLOPs. Under balanced routing, an expert receives no token with probability $(1 - k/E)^T \approx e^{-Tk/E}$, so for $T \gtrsim 5E/k$ (about 40 tokens at $E = 32$, $k = 4$; miss probability below 1%) essentially every expert is touched: the forward pass reads N_{total} weights, not N_{act} . The backward pass reads them again for activation gradients and writes the weight gradients, giving $3\beta N_{\text{total}}$ bytes of weight traffic per microbatch

with $\beta = 2$ bytes (bf16), independent of T . The arithmetic intensity is therefore

$$\mathcal{I}(T) = \frac{6N_{\text{act}}T}{3\beta N_{\text{total}}} = \frac{N_{\text{act}}}{N_{\text{total}}} T \quad \text{FLOP/byte}, \quad (2)$$

and the microbatch that reaches the ridge point is

$$T_{\min} = \frac{N_{\text{total}}}{N_{\text{act}}} \mathcal{I}^*. \quad (3)$$

Equivalently, each expert must see about $\mathcal{I}^*$ tokens per microbatch, as a dense matrix would. Below $T_{\min}$ the step is bandwidth-bound and

$$\text{throughput} = \frac{T b_s}{3\beta N_{\text{total}}} \text{ tokens/s}, \quad (4)$$

which depends on N_{total} and not on N_{act} . Above $T_{\min}$ the step is compute-bound at $u\pi/(6N_{\text{act}})$ tokens/s for a model FLOP utilization u , which we assume ($u = 25\%$) and do not measure.

Two further terms are estimated separately. An optimizer step reads and writes fp32 master weights and two Adam moments and reads the gradient, about 30 bytes per parameter, i.e. $30N_{\text{total}}/b_s$ seconds per step, amortized over the T_{opt} tokens accumulated per step. The training state occupies 16 bytes per parameter (bf16 weights and gradients, fp32 master weights and two moments). The model omits activation traffic, DeltaNet states, codec convolutions, MoE dispatch, kernel-launch overhead and multi-GPU communication, so it bounds throughput from above.

2.9 Ablation protocol

All ablations use `kairos overfit` on the configurations of `configs/ablations/` (`kairos==0.4.1.dev46`, `torch==2.14.0`), on CPU with eager attention and the PyTorch fallback DeltaNet path. The data are 16 identical copies of one 440-byte string (the sentence “the quick brown fox jumps over the lazy dog” repeated ten times), truncated to 128 tokens, i.e. about 2.9 repetitions per sequence; the 63-sentence control uses 63 distinct short sentences. Training uses batch size 4, 600 steps unless noted, $p_{\max} = 0.3$ and no reweighting; the attention window is the library default $w = 128$, which exceeds the 26 positions of the finest scale, so the window branch attends globally. Every step draws a fresh corruption, and *min. loss* is the minimum over the run of the per-step training loss, an extreme value of a noisy per-batch quantity. Training and evaluation data coincide, so the study measures memorization speed, not generalization. Runs are not bit-reproducible even at a fixed seed: the baseline at seed 0 gave 0.227 in one run and 0.31 in another. Seeds are 0, 1, 2 where stated; standard deviations are sample standard deviations ($n - 1$).

3 Results

Two unrelated kinds of result follow, at two scales: §3.1 is an analytical sizing of the untrained 200M/25M target, and §3.2 is measured data from the ablation networks. Neither substitutes for the other.

3.1 Compute sizing of the target design point

The target needs $C = 6N_{\text{act}}D = 6 \times 25\text{M} \times 30\text{B} = 4.5 \times 10^{18}$ FLOPs. Table 2 gives, for three GPUs, the ridge point, the smallest microbatch that hides the weight traffic of a dense model and of the sparse target ($N_{\text{total}}/N_{\text{act}} = 8$), and the compute-bound throughput and duration at the assumed 25% utilization (Figure 3).

A microbatch of about 1.6k tokens (T4, A100) to 2.4k tokens (H100) is enough for the sparse target to be compute-bound, against about 200–300 tokens for a dense model of the same size. At 2^{17} tokens per optimizer step, the memory-bound optimizer update adds less than 2.3% to the compute time on all

GPU	π (TFLOPS)	b_s (GB/s)	$\mathcal{I}^*$	$T_{\min}^{\text{dense}}$	$T_{\min}^{\text{MoE}}$	tok/s	Days
T4	65	320	203	203	1,625	108,333	3.21
A100 40GB	312	1,555	201	201	1,605	520,000	0.67
H100 SXM5	989	3,350	295	295	2,362	1,648,333	0.21

Table 2: Compute-bound sizing of the target ($N_{\text{total}} = 200\text{M}$, $N_{\text{act}} = 25\text{M}$, $D = 30\text{B}$). $T_{\min}$ is in tokens per microbatch (Eq. 3); tok/s and days assume 25% utilization, which is not measured. Peak dense half-precision throughput π and bandwidth b_s are vendor figures [???]. The table is generated by `scripts/roofline.py`.

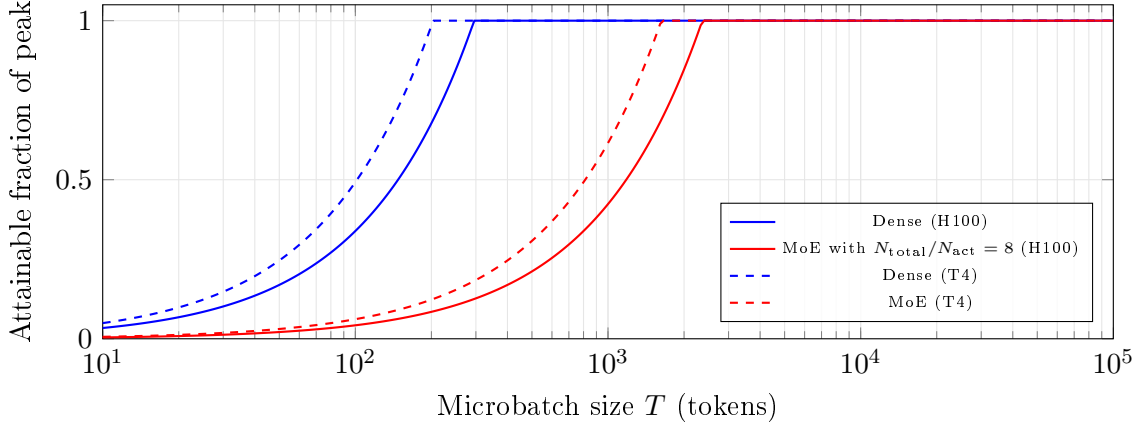


Figure 3: Model output (Eq. 2), not a measurement: the fraction of peak throughput a microbatch can reach. Sparsity moves the saturation point right by $N_{\text{total}}/N_{\text{act}} = 8$.

three GPUs. The 200M-parameter training state takes 3.2 GB and fits a 16 GB card, leaving the rest for activations, which set the achievable T . If activations or latency cap T below $T_{\min}$, the step time is set by N_{total} (Eq. 4); reducing N_{total} to 50M at fixed N_{act} divides $T_{\min}$ by four (about 400–600 tokens).

The target’s ratio $D/N_{\text{act}} = 1,200$ is below that of the dense Pythia-14M and Pythia-31M, trained on about 300B tokens ($\sim 21,000\times$ and $\sim 9,700\times$ their parameters) [?]. Those models are dense, so they bear on the plausibility of a high token-to-parameter ratio at small scale, not on sparse training.

3.2 Tiny-scale ablation study

Table 3 reports the codec and attention choices under three seeds, on the setup of §2.9. Codec ranking is identical on every seed: `patch` < `dense conv` < `conv` with 8 channels per group < `depthwise conv` (`lizz2` baseline), from 0.018 to 0.354. The hybrid attention beats SWA alone on every seed (0.227, 0.258, 0.576 against 0.596, 0.728, 0.798), although the mean $\pm$ sd ranges overlap. The PoC uses `conv` with 8 channels per group, a compromise between the depthwise default and the dense variants.

The seed spread (± 0.03 to ± 0.19) sets the noise floor for the single-run sweeps of Table 4, so differences of a few hundredths there are not readable. Monotone trends are consistent with the expected direction but confounded with the setup: at 128 tokens a sequence holds about 2.9 verbatim repetitions of the sentence (1.5 at 64, 5.8 at 256), so sequence length and the number of codec scales change how many copies are available for in-context recovery of masked bytes, and the loss reflects copying as much as memorization. A batch of 16 covers the whole training set. The attention-block-size result is a saturation effect: sizes at or above the number of layers give the same computation.

Configuration	Seed 0	Seed 1	Seed 2	Mean $\pm$ sd	Params
liz2 , depthwise codec (baseline)	0.227	0.258	0.576	0.354 ± 0.193	1.98M
vanilla (SWA only), depthwise codec	0.596	0.728	0.798	0.707 ± 0.103	1.81M
conv codec, 8 channels per group	0.106	0.045	0.075	0.075 ± 0.031	2.50M
conv codec, dense (64 channels per group)	0.037	0.024	0.024	0.028 ± 0.007	8.10M
patch codec (one Linear per scale)	0.019	0.016	0.018	0.018 ± 0.002	4.93M

Table 3: Minimum training loss after 600 steps, three seeds (`docs/experiments/ablations_codec_attention.md`, §1). Depthwise saves parameters relative to **patch** (1.98M against 4.93M) at the cost of slower memorization; 8 channels per group recovers most of the gap for about half of **patch**’s parameters (2.50M against 4.93M). The **liz2**-versus-**vanilla** gap is a convergence-speed effect (§3.2.1).

Factor (base configuration)	Min. loss by setting (single run)
Depth n_{layers} (liz2 baseline)	1: 0.227; 2: 0.843; 4: 0.368; 8 [†] : 1.228
MoE routing, 7 experts	top-1: 0.281; top-2: 0.272
Reweighting clip c (diffusion regime, conv , 8 channels per group)	$c = 3$: 0.139; $c = 30$: 0.249
AttnRes block size (4 layers)	4: 0.386; 8: 0.386; 16: 0.386
Codec scales S	2: 0.430; 3: 0.308; 4: 0.227
Sequence length (tokens)	64: 1.129; 128: 0.227; 256: 0.035
Batch size	1: 0.890; 4: 0.227; 16 (whole set): 0.032

Table 4: Single-run sweeps; read as indicative (§1 of the log gives the noise floor). [†]300 steps because of compute budget. The default $c = 10$ is omitted: the log has no per-run record for it. Depth does not help memorization and top-1 and top-2 routing are indistinguishable at this noise level.

3.2.1 A correlation-versus-causation case: shared versus separate QKV

Sharing the Q/K/V/O projections between the two attention branches (§2.3) first looked like a clear architectural win: with sharing disabled, min. loss rose from 0.227–0.369 to 0.610–0.858 across repeated 600-step runs. Three controls were run before accepting this as an architectural effect (Table 5).

The same check applies to **liz2** against **vanilla**: at 3000 steps **vanilla** reaches 0.0014, matching **liz2**’s 0.0011. Computing steps-to-threshold from the saved curves shows that the shared-versus-separate gap is concentrated in the 0.5–0.1 loss range (+9 to +14% steps for separate projections) and vanishes elsewhere, which a single comparison at a fixed step count would have hidden. A plausible mechanism is that shared projections receive gradient from both branches at every step, an effective higher early learning rate. The freezing control is consistent with it but does not isolate it; a direct test would rescale the learning rate of the separate variant. The lesson generalizes: a single-run result on a tiny memorization task is a hypothesis until it survives a step-budget control, a data-scale control and, where a mechanism is suspected, a control that isolates it.

3.2.2 Curriculum and memory gate

Table 6 splits one 600-step run (baseline, seed 0) by curriculum stage (200 steps each). Losses are not comparable across stages because the objective changes with (p_{max}, α) ; the higher tail mean of the transition stage is consistent with the reweighting ramp raising the loss scale. This is a single seed, and no run without the curriculum exists.

The memory gate was tested through the full training loop for 3 epochs (12 optimizer steps, so absolute losses are not comparable to the rest of the study): min. loss 7.512 without the gate, 7.453 with the previous batch’s state only, 7.423 with a random bank, and final loss 12.927, 12.893 and 9.752. Twelve steps cannot

Check	Shared	Separate	Outcome
600 steps, 16 identical sentences	0.369	0.610	shared ahead
3 000 steps, same data	0.0011	0.0011	gap closes
63 distinct sentences, 600 steps	2.269	2.285	gap closes
QKV frozen after step 100	0.705	0.521	gap reverses

Table 5: Min. loss for shared and separate projections (`docs/experiments/ablations_codec_attention.md`, §7; per-step curves in `docs/experiments/liz2_shared_vs_separate_followup_raw.jsonl`). The advantage was faster early convergence, not higher capacity: with enough steps or diverse data both variants reach the same floor.

Stage	First-step loss	Tail-mean loss
MAE ($p_{\max} = 0.3$, $\alpha = 0$)	8.663	1.485
Transition (linear ramp)	1.419	2.007
Diffusion ($p_{\max} = 1.0$, $\alpha = 1$)	1.873	1.555

Table 6: Per-stage loss of a single run (`docs/experiments/planned_experiments.md`, item 13); the tail definition is given there.

separate the variants; we draw no conclusion.

4 Discussion

Compute sizing. Under the model of §2.8, a sparse model trains at compute-bound speed only when its microbatch exceeds $N_{\text{total}}/N_{\text{act}}$ times the dense threshold; below that, throughput is set by N_{total} , not by the active parameters that FLOP-centric accounting counts. Sparsity is therefore not free on memory-limited hardware: E/k multiplies the microbatch needed for compute-bound execution, and activation memory grows with the microbatch. Whether a 16 GB card holds a 2.4k-token microbatch of this architecture is an empirical question this report does not answer.

Ablations. The only architectural comparisons tested at a longer horizon (`liz2` against `vanilla`, shared against separate projections) show no capacity difference. The codec ranking of Table 3 is stable across seeds but has not been tested at longer horizons or on diverse data, and should be read as a memorization-speed ranking.

4.1 Limitations

The target-scale analysis is a sizing model for a run that has not been performed: throughput is an upper bound at an assumed utilization, it assumes balanced routing and counts weight and optimizer traffic only, and the Chinchilla comparison is an extrapolation. The ablation study memorizes one sentence in 16 copies on text only, with a window branch that attends globally, on CPU, with runs that are not bit-reproducible; its metric is a minimum over noisy per-step losses. Only Table 3 and §3.2.1 carry multi-seed or multi-control statistics; the other sweeps are single-run. Parameter counts of the ablation rows in Table 3 other than the baseline are carried over from earlier logs. The PoC’s active-parameter count has not been logged from an instantiated model. No dense baseline is compared, so nothing here supports a claim of superiority; the multimodal, curriculum and memory-gate components have no supporting result yet.

4.2 Future work

Four experiments would turn this report’s hypotheses into findings: (i) a measured tokens/s against microbatch size for the 200M/25M and a 50M/25M variant on a T4 and an A100/H100, which tests Eqs. 3 and 4 directly; (ii) 3-seed, steps-to-threshold reruns of the single-run sweeps on diverse data; (iii) per-modality loss on real multimodal data; (iv) a run without the curriculum.

5 Conclusion

Kairos is a proof of concept of 1–15M parameters that runs a hybrid bidirectional DeltaNet/window-attention, MoE, denoising architecture end to end over a byte-level multimodal space, with a separate 200M/25M design point used only to size a future run. For that design point, sparse training is compute-bound once each expert sees about a ridge point’s worth of tokens per microbatch, roughly 1.6k–2.4k tokens; the ablation study’s main result is methodological, since its clearest architectural effect was a convergence-speed artifact.

Code and reproducibility

Code, configurations and logs are in the repository. Ablation configurations are in `configs/ablations/`; a run is `kairos overfit configs/ablations/<name>.yaml -steps 600 -mask-p-max 0.3 -no-mask-reweight -seed <0|1|2>`. The compute table is produced by `python scripts/roofline.py > docs/paper/tables/compute.tex` and checked by `tests/test_roofline.py` and `tests/test_paper.py`.